

CoSim Toolbox - Making HELICS Co-Simulations Easier

Trevor Hardy^{1*}, Mitch Pelton^{1*}, Nathan Gray^{1*}, Fred Rutz¹, and Wen Bo¹

¹ Pacific Northwest National Laboratory  * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

Co-simulation is a technique for creating larger and more complex time-series system models by facilitating data exchange between two or more models during runtime. This data exchange allows boundary states in models which may have been defined through static values or historical values to be dynamically generated by models in other simulator instances. This dynamic data exchange allows for system models that transcend individual simulation tools, modeled domains, and modeled subsystems. FMI ([FMI, 2025](#)), particularly as implemented in Modelon, is an example of a popular co-simulation platform.

HELICS ([Hardy et al., 2024](#)) is the US Department of Energy's publicly available co-simulation platform and has been under development for over eight years. HELICS provides the necessary simulation time management and facilitates the data exchanges between simulator instances (called "federates") and has been successfully used in a number of published studies such as [[Theisen:2024aa](#)], [[Bharati:2023aa](#)], [[Panossian:2023aa](#)] and [[Lardier:2020aa](#)]. Though these studies largely focus on power system applications, HELICS itself is domain agnostic.

CoSim Toolbox (CST) is built on top of HELICS and has the goal of making HELICS-based co-simulations easier to manage. Based on years of experience using HELICS, we have found a number of pain points around data and metadata management that CoSim Toolbox is designed to alleviate. Specifically, we have integrated the following tools and code to make using HELICS easier.

- **Docker** - Docker containers are used to distribute HELICS, the CST API as well as any CST-enabled simulation tools. This allows the use of CST on all Linux, MacOS, and Windows while minimizing the maintenance of these codebases.
- **Generic Federate class** - To facilitate writing new federates more easily, CST provides a generic federate class for use when writing new Python federates. This class wraps the HELICS APIs and facilitates data exchanges using just Python data structures. By minimizing the amount of knowledge of HELICS APIs, we have seen those with no knowledge of HELICS be able to more easily implement a HELICS-based co-simulation using this CST federate class.
- **Time-Series Data Collection** - A Postgres database is used to collect time-series data being generated by every federate. A data collection federate has been written using the CST APIs to collect data generated by the HELICS-enabled data exchanges and automatically commit it to the Postgres database. This allows data to be stored and thus accessed in a central location, making it easier to monitor and post-process the co-simulation data. The Postgres database is implemented as a Docker container along with pgAdmin to provide a web interface for inspecting the Postgres database.
- **Metadata and Configuration Management** - HELICS federates require significant configuration which has historically been accomplished by JSON files on disk. Particularly for larger federations, this leads to a significant number of JSON files, sometimes leading to

44 confusion about which files correspond to the data produced in a given co-simulation
45 run. To help manage this configuration data, CST uses MongoDB to store and the
46 configuration information with the CST federate class designed to automatically retrieve
47 a given federates configuration. The MongoDB database is implemented as a Docker
48 container along with Mongo Express to provide a a web interface for inspecting the
49 MongoDB database.

50 FMI, M. A. P. (2025). Fmi: Functional mock-up interface. In *GitHub repository*. GitHub.
51 <https://github.com/modelica/fmi-standard>

52 Hardy, T. D., Palmintier, B., Top, P. L., Krishnamurthy, D., & Fuller, J. C. (2024). HELICS: A
53 Co-Simulation Framework for Scalable Multi-Domain Modeling and Analysis. *IEEE Access*,
54 12, 24325–24347. <https://doi.org/10.1109/ACCESS.2024.3363615>

DRAFT